

# Track B · Vibe coding: build the same app by describing it

---

Session 5 · BDS M1 · Era 3 · ~20 minutes · work in pairs

Track A took six steps and seventy lines of Python. This takes one paragraph of English. Same app shape: pick some series, pick a window, compare, take the data home.

Nothing to install. A Google account is all you need.

**The rule:** describe the **outcome**, not the implementation. Do not name React, a database, or a caching trick. Two sentences in Step 1 break this rule on purpose — the reason is explained there.

---

## Step 0 · Open the Build tab · 1 min

[aistudio.google.com](https://aistudio.google.com) → sign in → **Build** in the left sidebar.

A prompt box, and a live preview once you submit. That is the whole environment.

---

## Step 1 · One paragraph, one app · 3 min

Paste this and hit enter:

Build a web app that compares currencies over time. Let me pick two or more currencies against the euro, pick a start and end date, and see them on one line chart, rebased so every line starts at 100 on my start date. Show me which currency gained the most. Use the Frankfurter exchange-rate service at <https://api.frankfurter.dev> — free, no API key. Fetch the rates on the server, not in the browser: the page asks my own app, and my app asks Frankfurter. Cache the result on the server. Organise the project into two folders: `backend/` and `frontend/`. Let me download the data as a CSV.

It builds in a minute or two.

**Why those server sentences are allowed.** They name outcomes, not technologies:

- **One fetch serves everyone.** That is Track A's `@st.cache_data`, one layer down.
- **A key would have somewhere to live.** Anything in the browser is readable by anyone.

"Use React" is a preference. "Don't put my key in the browser" is a requirement.

---

## Step 2 · Find the two halves · 6 min

Open the **Code** tab. You should have a `backend/` folder and a `frontend/` folder.

**First, prove the split is real.** Press **F12** → **Network** → reload. You should see a request to your own `/api/...`. If you see `api.frankfurter.dev` instead, everything is in the browser and there is no backend — say exactly that in the chat and ask again.

### The four connections

Line numbers below are from one real build. Yours will differ — **search for the strings**, not the numbers.

**1 • Frontend asks the backend** — `frontend/App.tsx`, line 84

```
const res = await fetch(`/api/rates?${params.toString()}`);
```

Note the leading slash. No domain, so the request never leaves your app.

**2 • Backend hears it** — `backend/server.ts`, line 37

```
app.get('/api/rates', async (req, res) => {
```

The same string on the other side. That match is the contract.

**3 • Backend asks Frankfurter** — `backend/server.ts` line 79, then `backend/frankfurter.ts` lines 98–100

```
const result = await getHistoricalRates(startDate, endDate, symbols);

const url = `${FRANKFURTER_BASE_URL}/${startDate}..${endDate}?
from=EUR&to=${symbolsParam}`;
const response = await fetch(url, { ... });
```

Only now does anything leave your app. This is the one file that talks to the outside world.

**4 • Backend answers, frontend draws** — `backend/server.ts` line 85, then `frontend/App.tsx` lines 90–91

```
return res.json({ ...result.data, cached: result.cached });

const data = await res.json();
setRatesData(data);
```

## The whole journey

```
App.tsx      fetch('/api/rates')
  ↓
server.ts    app.get('/api/rates')
  ↓
frankfurter.ts  fetch(Frankfurter)
  ↓
server.ts    res.json(...)
  ↓
App.tsx      res.json() → chart
```

**Remember it as:** the frontend asks through an API route, the backend does the work, the frontend shows the answer.

**Two questions for your pair:** if you added a paid data source tomorrow, which half would hold the key? And which half can any visitor read by pressing F12?

## Step 3 · Change one thing at a time · 3 min

Add a button that pins the current view — currencies, date range, and the winner — to a table below the chart.

**Try it:** ask for three changes in one message instead. It usually gets two right and drops the third silently.

**The point:** this is Track A's pin button, the one that needed `st.session_state`. Here you never learn the concept exists.

## Step 4 · Point, don't describe · 2 min

Use **annotation mode**: highlight part of the running app and describe the change there. Try it on the chart — thicker lines, legend above.

Pointing beats describing a location in words. Streamlit has no equivalent.

## Step 5 · Is it correct? · 5 min

It looks finished. That is not the same thing. **This step is the actual exercise.**

1. **Check one number.** Open `https://api.frankfurter.dev/v2/rates?base=eur` and compare with your app.
2. **Check the rebasing.** Every line must start at exactly 100 on your start date. If one starts at 97, every comparison is wrong.
3. **Break it.** Set the end date before the start date. A real app explains; a demo shows a blank screen.

4. **Ask the backend directly.** Copy your `/api/...` URL from the Network tab into a new browser tab. If the numbers are wrong there, it is the backend; right there but wrong on screen, it is the frontend.

Write down what you find. You need it for the comparison.

## Step 6 · Publish · 2 min

**Publish** (top right) → **Get Started** → **Publish App**. You get a live Cloud Run URL.

Free tier: two apps, no credit card. **Catch:** if your Google account has ever had Cloud billing attached, you are not eligible — use a personal account.

## Then compare — this is the assessment

Answer as a pair, in writing:

1. **Which was faster** to a first working version? Count Track A's setup, and Step 5 here.
2. **Which could you debug at 9am tomorrow** when someone says the numbers look wrong?
3. **What did Step 5 find?** If nothing, did you look — or did you trust the screen?
4. **Which would you put your name on** — and does that change if it will be used every week?

There is no right answer. There is a wrong one: assuming the newest tool is the best one.

## When re-prompting stops working

| SYMPTOM                            | TRY                                                 |
|------------------------------------|-----------------------------------------------------|
| Fixes one thing, breaks another    | One change per message; say what must stay the same |
| Insists it is fixed when it is not | Describe what you clicked, expected, and saw        |
| Keeps missing the same thing       | Open the Code tab and quote the relevant lines back |
| Everything is tangled              | Start a fresh chat. Cheaper than untangling         |

Every one of these works better the more code you understand. The 2025 study on your reading list found exactly that: expertise moves from writing lines to judging them.

## Building your own idea instead

- **Name your data.** "Use the Frankfurter API" works; "use some financial data" invents numbers that look real. Slide 18 lists four free, no-key APIs.
  - **Keep the server sentence.** It costs one line and gives you two halves to inspect in Step 2.
  - **One screen only.** Logins and saved accounts turn ten minutes into forty.
- 

## Appendix · prompts to paste

### Build:

Build a web app that compares currencies over time. Let me pick two or more currencies against the euro, pick a start and end date, and see them on one line chart, rebased so every line starts at 100 on my start date. Show me which currency gained the most. Use the Frankfurter exchange-rate service at <https://api.frankfurter.dev> — free, no API key. Fetch the rates on the server, not in the browser: the page asks my own app, and my app asks Frankfurter. Cache the result on the server. Organise the project into two folders: `backend/` and `frontend/`. Let me download the data as a CSV.

### If it built everything in the browser:

The page is calling `api.frankfurter.dev` directly from the browser. Move that call to the server: give the app its own endpoint that the page requests, and let only the server talk to Frankfurter.

### If the files came out flat:

Reorganise into two top-level folders, `backend/` and `frontend/`. Do not change any behaviour.

### Pin a view:

Add a button that pins the current view — currencies, date range, and the winner — to a table below the chart.

### Handle bad input:

If the end date is before the start date, show a clear message instead of an empty chart.